In [20]:
```
# ================================================================
# Label-efficiency with Copernicus-FM (frozen) for Water Mapping
# Fractions explored: {10, 20, 30, 50, 100} %
# Dataset: https://huggingface.co/datasets/giswqs/geospatial/resolve/main/waterb
# ================================================================

import os, math, time, zipfile, shutil, urllib.request, random, glob
from pathlib import Path
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

# ---- Copernicus-FM from torchgeo
from torchgeo.models import copernicusfm_base, CopernicusFM_Base_Weights

# ----------------------------- Repro -----------------------------
SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("device:", device)

# --------------------- Dataset (waterbody) -----------------------
def download_and_unzip(url, out_dir="waterbody_dataset"):
    out_dir = Path(out_dir)
    if (out_dir/"images").exists() and (out_dir/"masks").exists():
        print(f"Found dataset at: {out_dir}")
        return out_dir
    out_dir.mkdir(parents=True, exist_ok=True)
    zip_path = out_dir/"waterbody.zip"
    print("Downloading dataset...")
    urllib.request.urlretrieve(url, zip_path)
    print("Unzipping...")
    with zipfile.ZipFile(zip_path, "r") as zf:
        zf.extractall(out_dir)
    # bubble-up possible nested folders named images/ and masks/
    def bubble_up(folder_name):
        candidates = list(out_dir.rglob(folder_name))
        for c in candidates:
            if c.is_dir() and c.parent != out_dir:
                target = out_dir/folder_name
                target.mkdir(exist_ok=True)
                for p in c.glob("*"):
                    shutil.move(str(p), str(target/p.name))
        return out_dir/folder_name
    images = bubble_up("images"); masks = bubble_up("masks")
    assert images.exists() and masks.exists(), "images/ and masks/ not found aft
    zip_path.unlink(missing_ok=True)
    print(f"Dataset ready at: {out_dir}")
    return out_dir

URL = "https://huggingface.co/datasets/giswqs/geospatial/resolve/main/waterbody-
DATA_ROOT = download_and_unzip(URL, out_dir="waterbody_dataset")
IMG_DIR, MSK_DIR = Path(DATA_ROOT/"images"), Path(DATA_ROOT/"masks")
```

```
def list_pairs(img_dir, msk_dir, img_exts=("jpg","jpeg","png"), msk_exts=("jpg",
    imgs = []
    for e in img_exts: imgs += list(img_dir.glob(f"*.{e}"))
    imgs = sorted(imgs)
    mask_map = {}
    for e in msk_exts:
        for p in msk_dir.glob(f"*.{e}"): mask_map[p.stem] = p
    pairs=[]
    for ip in imgs:
        mp = mask_map.get(ip.stem, None)
        if mp is not None: pairs.append((ip, mp))
    return pairs

pairs = list_pairs(IMG_DIR, MSK_DIR)
print(f"Pairs found: {len(pairs)}")
assert len(pairs)>0, "No image/mask pairs."

# ------------------------- Preprocess ----------------------------
IMG_SIZE = 256   # must match encoder contract below

def letterbox_to_square(im_pil, mask_pil=None, size=IMG_SIZE):
    w,h = im_pil.size
    scale = size / min(w,h)
    nw, nh = max(1,int(round(w*scale))), max(1,int(round(h*scale)))
    im = im_pil.resize((nw, nh), Image.BILINEAR)
    mm = mask_pil.resize((nw, nh), Image.NEAREST) if mask_pil is not None else N
    # center crop/pad
    left = (nw-size)//2; top=(nh-size)//2; box=(left, top, left+size, top+size)
    if nw>=size and nh>=size:
        im = im.crop(box); mm = mm.crop(box) if mm is not None else None
    else:
        bg = Image.new("RGB",(size,size),(0,0,0)); bg.paste(im,((size-nw)//2,(si
        if mm is not None:
            bgm=Image.new("L",(size,size),0); bgm.paste(mm,((size-nw)//2,(size-n
    return im, mm

def compute_mean_std(paths, sample_k=800):
    sel = paths if len(paths)<=sample_k else random.sample(paths, sample_k)
    acc=np.zeros(3,np.float64); acc2=np.zeros(3,np.float64); n=0
    for p in sel:
        im = Image.open(p).convert("RGB")
        im,_ = letterbox_to_square(im,None,IMG_SIZE)
        arr = np.asarray(im,dtype=np.float32)/255.0
        m = arr.reshape(-1,3).mean(0); v=(arr.reshape(-1,3)**2).mean(0)
        acc += m; acc2 += v; n += 1
    mean=(acc/n).astype(np.float32); var=(acc2/n)-(mean**2); std=np.sqrt(np.clip
    return mean, std

# fixed split (val/test independent), mean/std from full train for stability
train_ratio, val_ratio = 0.8, 0.1
perm = np.random.permutation(len(pairs))
pairs = [pairs[i] for i in perm]
n=len(pairs); n_tr=int(round(n*train_ratio)); n_va=int(round(n*val_ratio))
train_pairs_full, val_pairs, test_pairs = pairs[:n_tr], pairs[n_tr:n_tr+n_va], p
mean_rgb, std_rgb = compute_mean_std([p for p,_ in train_pairs_full], 800)
print("mean:", np.round(mean_rgb,4), "std:", np.round(std_rgb,4))

class WaterDS(Dataset):
    def __init__(self, pairs, mean, std, augment=False):
```

```
        self.pairs=pairs; self.m=torch.tensor(mean).view(3,1,1); self.s=torch.te
        self.augment=augment
    def __len__(self): return len(self.pairs)
    def _norm(self,x): return (x-self.m)/(self.s+1e-6)
    def _aug(self,x,y):
        if random.random()<0.5: x=torch.flip(x,[2]); y=torch.flip(y,[1])
        if random.random()<0.5: x=torch.flip(x,[1]); y=torch.flip(y,[0])
        if random.random()<0.5:
            k=random.randint(0,3)
            if k: x=torch.rot90(x,k,[1,2]); y=torch.rot90(y,k,[0,1])
        return x,y
    def __getitem__(self,idx):
        ip,mp=self.pairs[idx]
        im = Image.open(ip).convert("RGB")
        mm = Image.open(mp).convert("L")
        im,mm = letterbox_to_square(im,mm,IMG_SIZE)
        img = torch.from_numpy(np.asarray(im,dtype=np.float32).transpose(2,0,1)/
        msk = torch.from_numpy((np.asarray(mm,dtype=np.uint8)>=128).astype(np.in
        img=self._norm(img)
        if self.augment: img,msk=self._aug(img,msk)
        return {"image":img, "mask":msk}

# ------------------- Encoder contract + wrapper -----------------
ENCODER_CONTRACT = {
    "model": "copernicusfm_base",
    "weights": "CopernicusFM_ViT",
    "input_mode": "spectral",         # we'll use patch_embed_spectral directly
    # torchgeo expects wavelengths/bandwidths in *nanometers*
    "wavelengths_nm": [480, 560, 650],
    "bandwidths_nm":  [50,  50,  50],
    "kernel_size": 16,                # patch size for tokenization
    "img_size": IMG_SIZE,
}

class CopFMSegWrapper(nn.Module):
    """
    Robust feature extractor: returns a 4D map [B,C,S,S].
    Uses patch_embed_spectral to avoid metadata requirements.
    """
    def __init__(self, enc, kernel_size, wvs_nm, bws_nm):
        super().__init__()
        self.enc = enc.eval()
        for p in self.enc.parameters(): p.requires_grad = False
        self.ks  = kernel_size
        self.register_buffer("wvs_nm", torch.tensor(wvs_nm, dtype=torch.float32)
        self.register_buffer("bws_nm", torch.tensor(bws_nm, dtype=torch.float32)
        if not hasattr(self.enc, "patch_embed_spectral"):
            raise AttributeError("CopernicusFM build has no 'patch_embed_spectra
    @staticmethod
    def _tokens_to_map(tokens: torch.Tensor) -> torch.Tensor:
        assert tokens.ndim == 3, f"Expected [B,N,D], got {tokens.shape}"
        B, N, D = tokens.shape
        S = int(round(N ** 0.5))
        assert S * S == N, f"Token length {N} is not a perfect square"
        return tokens.reshape(B, S, S, D).permute(0, 3, 1, 2).contiguous()
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        with torch.no_grad():
            toks = self.enc.patch_embed_spectral(
                x,
                wavelengths=self.wvs_nm.to(x.device),
```

```
                bandwidths=self.bws_nm.to(x.device),
                kernel_size=self.ks,
            )  # [B,N,D]
        return self._tokens_to_map(toks)  # [B,D,S,S]

# Build frozen CopFM encoder & probe shape
copfm = copernicusfm_base(weights=CopernicusFM_Base_Weights.CopernicusFM_ViT)
encoder = CopFMSegWrapper(
    copfm,
    kernel_size=ENCODER_CONTRACT["kernel_size"],
    wvs_nm=ENCODER_CONTRACT["wavelengths_nm"],
    bws_nm=ENCODER_CONTRACT["bandwidths_nm"],
).to(device).eval()
for p in encoder.parameters(): p.requires_grad=False

with torch.no_grad():
    probe = torch.zeros(2,3,IMG_SIZE,IMG_SIZE,device=device)
    feat  = encoder(probe)   # [2,C,S,S]
C_ENC, S_ENC = feat.shape[1], feat.shape[2]
print(f"Encoder feature: C={C_ENC}, S={S_ENC} (IMG={IMG_SIZE}, patch_k={ENCODER_

# ------------------------- Small decoder -------------------------
def GN(ch):
    g = 32 if ch>=32 else max(1, ch//2)
    return nn.GroupNorm(g, ch)

NUM_CLASSES = 2
class SmallDecoder(nn.Module):
    def __init__(self, in_ch, num_classes=NUM_CLASSES, out_size=(IMG_SIZE,IMG_SI
        super().__init__()
        m1, m2 = in_ch//2, in_ch//4
        self.net = nn.Sequential(
            nn.Conv2d(in_ch, m1, 3, padding=1), GN(m1), nn.ReLU(True), nn.Dropou
            nn.Conv2d(m1,  m2, 3, padding=1), GN(m2), nn.ReLU(True), nn.Dropout2
            nn.Conv2d(m2,  num_classes, 1),
        )
        self.out_size = out_size
    def forward(self, x):
        x = self.net(x)
        return F.interpolate(x, size=self.out_size, mode="bilinear", align_corne

# ---------------------- Train / Eval utils -----------------------
EPOCHS=100; LR=1e-4; WD=1e-4; USE_DICE=True
criterion = nn.CrossEntropyLoss()

@torch.no_grad()
def forward_feats(x):  # encoder is frozen
    return encoder(x)

def dice_loss_mc(logits, y, eps=1e-6, C=NUM_CLASSES):
    probs = torch.softmax(logits, dim=1)
    y1 = torch.zeros_like(probs).scatter_(1, y.unsqueeze(1), 1.0)
    inter = (probs*y1).sum(dim=(0,2,3))
    den   = (probs+y1).sum(dim=(0,2,3))
    dice  = (2*inter+eps)/(den+eps)
    return 1.0 - dice.mean()

def pixel_acc(logits, y):
    pred = logits.argmax(1)
    return (pred==y).sum().item(), y.numel()
```

```
@torch.no_grad()
def eval_metrics(decoder, loader):
    decoder.eval()
    corr=0; totpix=0
    inter=np.zeros(NUM_CLASSES,np.int64); union=np.zeros(NUM_CLASSES,np.int64)
    for b in loader:
        x=b["image"].to(device); y=b["mask"].to(device)
        f=forward_feats(x)
        pred=decoder(f).argmax(1)
        corr+=(pred==y).sum().item(); totpix+=y.numel()
        p=pred.cpu().numpy().ravel(); g=y.cpu().numpy().ravel()
        for c in range(NUM_CLASSES):
            pi=(p==c); gi=(g==c)
            tp = np.logical_and(pi,gi).sum()
            un = np.logical_or (pi,gi).sum()
            inter[c]+=tp; union[c]+=un
    oa = corr/max(1,totpix)
    iou_c = inter/np.maximum(1,union)
    dice_c = (2*inter)/np.maximum(1,2*inter + (union - inter))  # 2TP/(2TP+FP+FN
    return float(oa), float(iou_c.mean()), float(dice_c.mean())

def model_size_mb(model):
    # rough: float32 params only
    n_params = sum(p.numel() for p in model.parameters())
    return n_params*4/1e6

# ------------------------ Fractions loop -------------------------
FRACS = [10, 20, 30, 50, 100]  # percent
BATCH=16; WORKERS=0
RESULTS = []
SAVE_DIR = Path("copfm_label_eff_models"); SAVE_DIR.mkdir(exist_ok=True)

# fixed val/test loaders
val_dl   = DataLoader(WaterDS(val_pairs,  mean_rgb,std_rgb,augment=False), batch
test_dl  = DataLoader(WaterDS(test_pairs, mean_rgb,std_rgb,augment=False), batch

for frac in FRACS:
    # subset of train pairs
    k = max(1, int(len(train_pairs_full)*frac/100.0))
    train_pairs = train_pairs_full[:k]  # deterministic by prior permutation
    train_dl = DataLoader(WaterDS(train_pairs, mean_rgb, std_rgb, augment=True),
                          batch_size=BATCH, shuffle=True, num_workers=WORKERS, d

    # fresh decoder
    decoder = SmallDecoder(in_ch=C_ENC).to(device)
    opt = torch.optim.AdamW(decoder.parameters(), lr=LR, weight_decay=WD)

    # log counts/params
    params_total = sum(p.numel() for p in decoder.parameters())
    params_trainable = sum(p.numel() for p in decoder.parameters() if p.requires
    ms_mb = model_size_mb(decoder)

    # quick inference timing probe (decoder only, encoder frozen)
    decoder.eval()
    xb = next(iter(train_dl))["image"].to(device)
    with torch.no_grad():
        fb = forward_feats(xb)
        t0=time.time()
        for _ in range(10):
```

```
            _ = decoder(fb)
        torch.cuda.synchronize() if torch.cuda.is_available() else None
        infer_ms = (time.time()-t0)/10*1000.0/xb.shape[0]  # per image

    # training
    print(f"\n=== Fraction {frac}% | train images: {k} ===")
    t_train0 = time.time()
    for ep in range(1, EPOCHS+1):
        decoder.train()
        tot=0.0; corr=0; totpix=0
        for b in train_dl:
            x=b["image"].to(device); y=b["mask"].to(device)
            with torch.no_grad(): f=forward_feats(x)
            logits = decoder(f)
            loss = criterion(logits,y) + (dice_loss_mc(logits,y) if USE_DICE els
            opt.zero_grad(); loss.backward(); opt.step()
            c,t = pixel_acc(logits,y); corr+=c; totpix+=t
            tot += float(loss.item())
        tr_loss = tot/max(1,len(train_dl)); tr_oa = corr/max(1,totpix)
        va_oa, va_iou, va_dice = eval_metrics(decoder, val_dl)
        print(f"Epoch {ep:02d} | loss {tr_loss:.4f} | val_OA {va_oa:.4f} | val_I
    train_time_s = time.time()-t_train0

    # final metrics (val + test)
    val_OA, val_IoU, val_Dice = eval_metrics(decoder, val_dl)
    test_OA, test_IoU, test_Dice = eval_metrics(decoder, test_dl)

    # save decoder only (safe fields)
    ckpt_path = SAVE_DIR/f"copfm_frac_{frac:02d}.pth"
    torch.save({
        "decoder_state": decoder.state_dict(),
        "in_ch": C_ENC,
        "num_classes": NUM_CLASSES,
        "img_size": IMG_SIZE,
        "encoder_contract": ENCODER_CONTRACT,  # for traceability
    }, ckpt_path)

    RESULTS.append({
        "fraction_pct": frac,
        "n_train_imgs": k,
        "params_total": params_total,
        "params_trainable": params_trainable,
        "model_size_MB": ms_mb,
        "train_time_s": train_time_s,
        "infer_ms_per_img": infer_ms,
        "val_OA": val_OA, "val_IoU": val_IoU, "val_Dice": val_Dice,
        "test_OA": test_OA, "test_IoU": test_IoU, "test_Dice": test_Dice,
        "ckpt": str(ckpt_path),
    })
```

```
device: cuda
Found dataset at: waterbody_dataset
Pairs found: 2841
mean: [0.3407 0.3721 0.4114] std: [0.1849 0.1489 0.1516]
Encoder feature: C=768, S=16 (IMG=256, patch_k=16)

=== Fraction 10% | train images: 227 ===
Epoch 01 | loss 1.0422 | val_OA 0.7537 | val_IoU 0.5769 | val_Dice 0.7257
Epoch 02 | loss 0.8773 | val_OA 0.7590 | val_IoU 0.5884 | val_Dice 0.7361
Epoch 03 | loss 0.8620 | val_OA 0.7549 | val_IoU 0.5561 | val_Dice 0.7028
Epoch 04 | loss 0.8246 | val_OA 0.7518 | val_IoU 0.5696 | val_Dice 0.7185
Epoch 05 | loss 0.7948 | val_OA 0.7560 | val_IoU 0.5809 | val_Dice 0.7292
Epoch 06 | loss 0.7900 | val_OA 0.7659 | val_IoU 0.5961 | val_Dice 0.7421
Epoch 07 | loss 0.7786 | val_OA 0.7720 | val_IoU 0.6147 | val_Dice 0.7589
Epoch 08 | loss 0.7539 | val_OA 0.7800 | val_IoU 0.6223 | val_Dice 0.7641
Epoch 09 | loss 0.7300 | val_OA 0.7921 | val_IoU 0.6298 | val_Dice 0.7684
Epoch 10 | loss 0.7251 | val_OA 0.7910 | val_IoU 0.6361 | val_Dice 0.7745
Epoch 11 | loss 0.7446 | val_OA 0.7639 | val_IoU 0.5923 | val_Dice 0.7387
Epoch 12 | loss 0.7368 | val_OA 0.7669 | val_IoU 0.5936 | val_Dice 0.7392
Epoch 13 | loss 0.7364 | val_OA 0.7655 | val_IoU 0.5832 | val_Dice 0.7289
Epoch 14 | loss 0.7052 | val_OA 0.7924 | val_IoU 0.6275 | val_Dice 0.7662
Epoch 15 | loss 0.6892 | val_OA 0.8020 | val_IoU 0.6490 | val_Dice 0.7839
Epoch 16 | loss 0.6740 | val_OA 0.7925 | val_IoU 0.6438 | val_Dice 0.7813
Epoch 17 | loss 0.6825 | val_OA 0.7897 | val_IoU 0.6383 | val_Dice 0.7769
Epoch 18 | loss 0.6974 | val_OA 0.8076 | val_IoU 0.6551 | val_Dice 0.7882
Epoch 19 | loss 0.6735 | val_OA 0.7996 | val_IoU 0.6326 | val_Dice 0.7693
Epoch 20 | loss 0.6612 | val_OA 0.8027 | val_IoU 0.6497 | val_Dice 0.7844
Epoch 21 | loss 0.6827 | val_OA 0.7974 | val_IoU 0.6410 | val_Dice 0.7776
Epoch 22 | loss 0.6659 | val_OA 0.7992 | val_IoU 0.6456 | val_Dice 0.7815
Epoch 23 | loss 0.6427 | val_OA 0.8004 | val_IoU 0.6543 | val_Dice 0.7890
Epoch 24 | loss 0.6603 | val_OA 0.8091 | val_IoU 0.6626 | val_Dice 0.7946
Epoch 25 | loss 0.6328 | val_OA 0.8043 | val_IoU 0.6536 | val_Dice 0.7876
Epoch 26 | loss 0.6479 | val_OA 0.8157 | val_IoU 0.6686 | val_Dice 0.7985
Epoch 27 | loss 0.6529 | val_OA 0.8060 | val_IoU 0.6454 | val_Dice 0.7798
Epoch 28 | loss 0.6187 | val_OA 0.8031 | val_IoU 0.6572 | val_Dice 0.7911
Epoch 29 | loss 0.6099 | val_OA 0.8100 | val_IoU 0.6655 | val_Dice 0.7970
Epoch 30 | loss 0.6365 | val_OA 0.8058 | val_IoU 0.6503 | val_Dice 0.7843
Epoch 31 | loss 0.6353 | val_OA 0.8006 | val_IoU 0.6431 | val_Dice 0.7788
Epoch 32 | loss 0.6129 | val_OA 0.8145 | val_IoU 0.6700 | val_Dice 0.7999
Epoch 33 | loss 0.6289 | val_OA 0.8130 | val_IoU 0.6701 | val_Dice 0.8004
Epoch 34 | loss 0.6148 | val_OA 0.8192 | val_IoU 0.6773 | val_Dice 0.8054
Epoch 35 | loss 0.5830 | val_OA 0.8205 | val_IoU 0.6718 | val_Dice 0.8003
Epoch 36 | loss 0.6251 | val_OA 0.8149 | val_IoU 0.6702 | val_Dice 0.8000
Epoch 37 | loss 0.6437 | val_OA 0.8030 | val_IoU 0.6603 | val_Dice 0.7938
Epoch 38 | loss 0.6210 | val_OA 0.8121 | val_IoU 0.6645 | val_Dice 0.7956
Epoch 39 | loss 0.6211 | val_OA 0.8167 | val_IoU 0.6716 | val_Dice 0.8009
Epoch 40 | loss 0.5921 | val_OA 0.8155 | val_IoU 0.6727 | val_Dice 0.8021
Epoch 41 | loss 0.6046 | val_OA 0.8120 | val_IoU 0.6680 | val_Dice 0.7987
Epoch 42 | loss 0.5797 | val_OA 0.8220 | val_IoU 0.6782 | val_Dice 0.8055
Epoch 43 | loss 0.5875 | val_OA 0.8195 | val_IoU 0.6781 | val_Dice 0.8060
Epoch 44 | loss 0.6011 | val_OA 0.8190 | val_IoU 0.6719 | val_Dice 0.8007
Epoch 45 | loss 0.5699 | val_OA 0.8124 | val_IoU 0.6560 | val_Dice 0.7880
Epoch 46 | loss 0.5836 | val_OA 0.8200 | val_IoU 0.6811 | val_Dice 0.8085
Epoch 47 | loss 0.5954 | val_OA 0.8169 | val_IoU 0.6692 | val_Dice 0.7988
Epoch 48 | loss 0.5720 | val_OA 0.8225 | val_IoU 0.6803 | val_Dice 0.8073
Epoch 49 | loss 0.5561 | val_OA 0.8177 | val_IoU 0.6712 | val_Dice 0.8004
Epoch 50 | loss 0.6281 | val_OA 0.8225 | val_IoU 0.6826 | val_Dice 0.8093
Epoch 51 | loss 0.5812 | val_OA 0.8181 | val_IoU 0.6720 | val_Dice 0.8010
Epoch 52 | loss 0.5738 | val_OA 0.8226 | val_IoU 0.6714 | val_Dice 0.7995
Epoch 53 | loss 0.5673 | val_OA 0.8240 | val_IoU 0.6783 | val_Dice 0.8053
```

```
Epoch 54 | loss 0.5567 | val_OA 0.8133 | val_IoU 0.6629 | val_Dice 0.7939
Epoch 55 | loss 0.5720 | val_OA 0.8227 | val_IoU 0.6748 | val_Dice 0.8025
Epoch 56 | loss 0.5643 | val_OA 0.8274 | val_IoU 0.6840 | val_Dice 0.8095
Epoch 57 | loss 0.6031 | val_OA 0.7990 | val_IoU 0.6329 | val_Dice 0.7697
Epoch 58 | loss 0.5995 | val_OA 0.8187 | val_IoU 0.6632 | val_Dice 0.7931
Epoch 59 | loss 0.5975 | val_OA 0.8233 | val_IoU 0.6837 | val_Dice 0.8100
Epoch 60 | loss 0.5497 | val_OA 0.8247 | val_IoU 0.6833 | val_Dice 0.8094
Epoch 61 | loss 0.5358 | val_OA 0.8264 | val_IoU 0.6868 | val_Dice 0.8120
Epoch 62 | loss 0.5374 | val_OA 0.8270 | val_IoU 0.6868 | val_Dice 0.8119
Epoch 63 | loss 0.5362 | val_OA 0.8217 | val_IoU 0.6754 | val_Dice 0.8032
Epoch 64 | loss 0.5698 | val_OA 0.8217 | val_IoU 0.6787 | val_Dice 0.8060
Epoch 65 | loss 0.5375 | val_OA 0.8205 | val_IoU 0.6727 | val_Dice 0.8011
Epoch 66 | loss 0.5710 | val_OA 0.8202 | val_IoU 0.6763 | val_Dice 0.8043
Epoch 67 | loss 0.5429 | val_OA 0.8213 | val_IoU 0.6817 | val_Dice 0.8087
Epoch 68 | loss 0.5413 | val_OA 0.8315 | val_IoU 0.6932 | val_Dice 0.8165
Epoch 69 | loss 0.5501 | val_OA 0.8293 | val_IoU 0.6933 | val_Dice 0.8169
Epoch 70 | loss 0.5617 | val_OA 0.8273 | val_IoU 0.6891 | val_Dice 0.8138
Epoch 71 | loss 0.5711 | val_OA 0.8275 | val_IoU 0.6890 | val_Dice 0.8137
Epoch 72 | loss 0.5407 | val_OA 0.8314 | val_IoU 0.6935 | val_Dice 0.8168
Epoch 73 | loss 0.5383 | val_OA 0.8308 | val_IoU 0.6866 | val_Dice 0.8110
Epoch 74 | loss 0.5297 | val_OA 0.8288 | val_IoU 0.6846 | val_Dice 0.8097
Epoch 75 | loss 0.5096 | val_OA 0.8314 | val_IoU 0.6919 | val_Dice 0.8154
Epoch 76 | loss 0.5161 | val_OA 0.8272 | val_IoU 0.6877 | val_Dice 0.8127
Epoch 77 | loss 0.5281 | val_OA 0.8267 | val_IoU 0.6841 | val_Dice 0.8097
Epoch 78 | loss 0.5333 | val_OA 0.8285 | val_IoU 0.6925 | val_Dice 0.8165
Epoch 79 | loss 0.5401 | val_OA 0.8096 | val_IoU 0.6706 | val_Dice 0.8014
Epoch 80 | loss 0.5620 | val_OA 0.8271 | val_IoU 0.6876 | val_Dice 0.8126
Epoch 81 | loss 0.5480 | val_OA 0.8317 | val_IoU 0.6930 | val_Dice 0.8163
Epoch 82 | loss 0.5259 | val_OA 0.8302 | val_IoU 0.6855 | val_Dice 0.8102
Epoch 83 | loss 0.5437 | val_OA 0.8121 | val_IoU 0.6503 | val_Dice 0.7829
Epoch 84 | loss 0.5561 | val_OA 0.8279 | val_IoU 0.6876 | val_Dice 0.8125
Epoch 85 | loss 0.5668 | val_OA 0.8298 | val_IoU 0.6935 | val_Dice 0.8170
Epoch 86 | loss 0.5235 | val_OA 0.8305 | val_IoU 0.6948 | val_Dice 0.8180
Epoch 87 | loss 0.5441 | val_OA 0.8329 | val_IoU 0.6956 | val_Dice 0.8182
Epoch 88 | loss 0.5205 | val_OA 0.8308 | val_IoU 0.6883 | val_Dice 0.8125
Epoch 89 | loss 0.4980 | val_OA 0.8266 | val_IoU 0.6894 | val_Dice 0.8142
Epoch 90 | loss 0.4911 | val_OA 0.8279 | val_IoU 0.6842 | val_Dice 0.8096
Epoch 91 | loss 0.5276 | val_OA 0.8309 | val_IoU 0.6946 | val_Dice 0.8178
Epoch 92 | loss 0.5082 | val_OA 0.8338 | val_IoU 0.6934 | val_Dice 0.8162
Epoch 93 | loss 0.5085 | val_OA 0.8303 | val_IoU 0.6900 | val_Dice 0.8140
Epoch 94 | loss 0.5126 | val_OA 0.8268 | val_IoU 0.6912 | val_Dice 0.8157
Epoch 95 | loss 0.5228 | val_OA 0.8314 | val_IoU 0.6868 | val_Dice 0.8111
Epoch 96 | loss 0.5171 | val_OA 0.8271 | val_IoU 0.6805 | val_Dice 0.8066
Epoch 97 | loss 0.5156 | val_OA 0.8321 | val_IoU 0.6932 | val_Dice 0.8164
Epoch 98 | loss 0.4963 | val_OA 0.8293 | val_IoU 0.6869 | val_Dice 0.8116
Epoch 99 | loss 0.4955 | val_OA 0.8340 | val_IoU 0.6929 | val_Dice 0.8157
Epoch 100 | loss 0.5104 | val_OA 0.8338 | val_IoU 0.6946 | val_Dice 0.8172

=== Fraction 20% | train images: 454 ===
Epoch 01 | loss 0.9296 | val_OA 0.7505 | val_IoU 0.5664 | val_Dice 0.7154
Epoch 02 | loss 0.8332 | val_OA 0.7843 | val_IoU 0.6248 | val_Dice 0.7655
Epoch 03 | loss 0.8130 | val_OA 0.7815 | val_IoU 0.6203 | val_Dice 0.7619
Epoch 04 | loss 0.7699 | val_OA 0.7860 | val_IoU 0.6217 | val_Dice 0.7621
Epoch 05 | loss 0.7761 | val_OA 0.7948 | val_IoU 0.6451 | val_Dice 0.7820
Epoch 06 | loss 0.7218 | val_OA 0.8062 | val_IoU 0.6545 | val_Dice 0.7880
Epoch 07 | loss 0.7016 | val_OA 0.8030 | val_IoU 0.6567 | val_Dice 0.7906
Epoch 08 | loss 0.7261 | val_OA 0.7849 | val_IoU 0.6016 | val_Dice 0.7426
Epoch 09 | loss 0.7273 | val_OA 0.8041 | val_IoU 0.6582 | val_Dice 0.7918
Epoch 10 | loss 0.6992 | val_OA 0.7986 | val_IoU 0.6435 | val_Dice 0.7797
Epoch 11 | loss 0.7112 | val_OA 0.8084 | val_IoU 0.6579 | val_Dice 0.7906
```

```
Epoch 12 | loss 0.6829 | val_OA 0.8144 | val_IoU 0.6731 | val_Dice 0.8026
Epoch 13 | loss 0.6816 | val_OA 0.8113 | val_IoU 0.6588 | val_Dice 0.7908
Epoch 14 | loss 0.6683 | val_OA 0.7791 | val_IoU 0.6085 | val_Dice 0.7510
Epoch 15 | loss 0.6722 | val_OA 0.7989 | val_IoU 0.6420 | val_Dice 0.7782
Epoch 16 | loss 0.6708 | val_OA 0.8160 | val_IoU 0.6761 | val_Dice 0.8050
Epoch 17 | loss 0.6632 | val_OA 0.8152 | val_IoU 0.6741 | val_Dice 0.8034
Epoch 18 | loss 0.6419 | val_OA 0.7931 | val_IoU 0.6275 | val_Dice 0.7660
Epoch 19 | loss 0.6781 | val_OA 0.8084 | val_IoU 0.6592 | val_Dice 0.7917
Epoch 20 | loss 0.6536 | val_OA 0.8084 | val_IoU 0.6554 | val_Dice 0.7884
Epoch 21 | loss 0.6478 | val_OA 0.8269 | val_IoU 0.6869 | val_Dice 0.8120
Epoch 22 | loss 0.6231 | val_OA 0.8175 | val_IoU 0.6743 | val_Dice 0.8031
Epoch 23 | loss 0.6194 | val_OA 0.8306 | val_IoU 0.6922 | val_Dice 0.8158
Epoch 24 | loss 0.6234 | val_OA 0.8309 | val_IoU 0.6950 | val_Dice 0.8181
Epoch 25 | loss 0.6189 | val_OA 0.8060 | val_IoU 0.6570 | val_Dice 0.7903
Epoch 26 | loss 0.6753 | val_OA 0.8247 | val_IoU 0.6867 | val_Dice 0.8123
Epoch 27 | loss 0.6114 | val_OA 0.8309 | val_IoU 0.6861 | val_Dice 0.8106
Epoch 28 | loss 0.6149 | val_OA 0.8298 | val_IoU 0.6934 | val_Dice 0.8170
Epoch 29 | loss 0.6108 | val_OA 0.8295 | val_IoU 0.6950 | val_Dice 0.8183
Epoch 30 | loss 0.5941 | val_OA 0.8350 | val_IoU 0.6937 | val_Dice 0.8162
Epoch 31 | loss 0.6117 | val_OA 0.8298 | val_IoU 0.6875 | val_Dice 0.8120
Epoch 32 | loss 0.5757 | val_OA 0.8391 | val_IoU 0.7016 | val_Dice 0.8221
Epoch 33 | loss 0.5801 | val_OA 0.8304 | val_IoU 0.6913 | val_Dice 0.8151
Epoch 34 | loss 0.5916 | val_OA 0.8389 | val_IoU 0.7063 | val_Dice 0.8259
Epoch 35 | loss 0.6000 | val_OA 0.8324 | val_IoU 0.6957 | val_Dice 0.8184
Epoch 36 | loss 0.6025 | val_OA 0.8410 | val_IoU 0.7066 | val_Dice 0.8259
Epoch 37 | loss 0.5749 | val_OA 0.8311 | val_IoU 0.6937 | val_Dice 0.8169
Epoch 38 | loss 0.5770 | val_OA 0.8146 | val_IoU 0.6573 | val_Dice 0.7887
Epoch 39 | loss 0.5869 | val_OA 0.8369 | val_IoU 0.7014 | val_Dice 0.8223
Epoch 40 | loss 0.5706 | val_OA 0.8408 | val_IoU 0.7076 | val_Dice 0.8267
Epoch 41 | loss 0.5701 | val_OA 0.8366 | val_IoU 0.7054 | val_Dice 0.8256
Epoch 42 | loss 0.5715 | val_OA 0.8396 | val_IoU 0.7047 | val_Dice 0.8245
Epoch 43 | loss 0.5593 | val_OA 0.8420 | val_IoU 0.7058 | val_Dice 0.8250
Epoch 44 | loss 0.5606 | val_OA 0.8433 | val_IoU 0.7142 | val_Dice 0.8316
Epoch 45 | loss 0.5568 | val_OA 0.8469 | val_IoU 0.7176 | val_Dice 0.8337
Epoch 46 | loss 0.5532 | val_OA 0.8418 | val_IoU 0.7081 | val_Dice 0.8269
Epoch 47 | loss 0.5688 | val_OA 0.8438 | val_IoU 0.7097 | val_Dice 0.8278
Epoch 48 | loss 0.5512 | val_OA 0.8474 | val_IoU 0.7180 | val_Dice 0.8339
Epoch 49 | loss 0.5567 | val_OA 0.8394 | val_IoU 0.7034 | val_Dice 0.8234
Epoch 50 | loss 0.5699 | val_OA 0.8454 | val_IoU 0.7135 | val_Dice 0.8306
Epoch 51 | loss 0.5527 | val_OA 0.8265 | val_IoU 0.6813 | val_Dice 0.8074
Epoch 52 | loss 0.5676 | val_OA 0.8470 | val_IoU 0.7172 | val_Dice 0.8334
Epoch 53 | loss 0.5485 | val_OA 0.8312 | val_IoU 0.6823 | val_Dice 0.8073
Epoch 54 | loss 0.5577 | val_OA 0.8434 | val_IoU 0.7166 | val_Dice 0.8335
Epoch 55 | loss 0.5400 | val_OA 0.8423 | val_IoU 0.7059 | val_Dice 0.8250
Epoch 56 | loss 0.5310 | val_OA 0.8472 | val_IoU 0.7172 | val_Dice 0.8333
Epoch 57 | loss 0.5345 | val_OA 0.8417 | val_IoU 0.7027 | val_Dice 0.8225
Epoch 58 | loss 0.5219 | val_OA 0.8496 | val_IoU 0.7244 | val_Dice 0.8386
Epoch 59 | loss 0.5296 | val_OA 0.8449 | val_IoU 0.7112 | val_Dice 0.8289
Epoch 60 | loss 0.5341 | val_OA 0.8421 | val_IoU 0.7039 | val_Dice 0.8234
Epoch 61 | loss 0.5261 | val_OA 0.8477 | val_IoU 0.7191 | val_Dice 0.8348
Epoch 62 | loss 0.5347 | val_OA 0.8431 | val_IoU 0.7088 | val_Dice 0.8273
Epoch 63 | loss 0.5263 | val_OA 0.8499 | val_IoU 0.7247 | val_Dice 0.8389
Epoch 64 | loss 0.5240 | val_OA 0.8476 | val_IoU 0.7167 | val_Dice 0.8329
Epoch 65 | loss 0.5365 | val_OA 0.8465 | val_IoU 0.7181 | val_Dice 0.8342
Epoch 66 | loss 0.5434 | val_OA 0.8350 | val_IoU 0.7005 | val_Dice 0.8219
Epoch 67 | loss 0.5222 | val_OA 0.8369 | val_IoU 0.7039 | val_Dice 0.8243
Epoch 68 | loss 0.5197 | val_OA 0.8510 | val_IoU 0.7226 | val_Dice 0.8370
Epoch 69 | loss 0.5327 | val_OA 0.8418 | val_IoU 0.7082 | val_Dice 0.8270
Epoch 70 | loss 0.5284 | val_OA 0.8516 | val_IoU 0.7269 | val_Dice 0.8403
Epoch 71 | loss 0.5222 | val_OA 0.8484 | val_IoU 0.7199 | val_Dice 0.8353
```

```
Epoch 72 | loss 0.5107 | val_OA 0.8459 | val_IoU 0.7152 | val_Dice 0.8320
Epoch 73 | loss 0.5169 | val_OA 0.8510 | val_IoU 0.7228 | val_Dice 0.8371
Epoch 74 | loss 0.5278 | val_OA 0.8515 | val_IoU 0.7223 | val_Dice 0.8367
Epoch 75 | loss 0.5158 | val_OA 0.8544 | val_IoU 0.7280 | val_Dice 0.8407
Epoch 76 | loss 0.5066 | val_OA 0.8424 | val_IoU 0.7144 | val_Dice 0.8319
Epoch 77 | loss 0.5300 | val_OA 0.8326 | val_IoU 0.6931 | val_Dice 0.8162
Epoch 78 | loss 0.5194 | val_OA 0.8436 | val_IoU 0.7198 | val_Dice 0.8360
Epoch 79 | loss 0.5249 | val_OA 0.8374 | val_IoU 0.6988 | val_Dice 0.8201
Epoch 80 | loss 0.5121 | val_OA 0.8537 | val_IoU 0.7284 | val_Dice 0.8411
Epoch 81 | loss 0.5074 | val_OA 0.8504 | val_IoU 0.7197 | val_Dice 0.8348
Epoch 82 | loss 0.4984 | val_OA 0.8503 | val_IoU 0.7227 | val_Dice 0.8372
Epoch 83 | loss 0.5076 | val_OA 0.8530 | val_IoU 0.7271 | val_Dice 0.8402
Epoch 84 | loss 0.4984 | val_OA 0.8481 | val_IoU 0.7138 | val_Dice 0.8304
Epoch 85 | loss 0.5058 | val_OA 0.8465 | val_IoU 0.7148 | val_Dice 0.8315
Epoch 86 | loss 0.5211 | val_OA 0.8445 | val_IoU 0.7160 | val_Dice 0.8329
Epoch 87 | loss 0.5146 | val_OA 0.8508 | val_IoU 0.7235 | val_Dice 0.8377
Epoch 88 | loss 0.5057 | val_OA 0.8500 | val_IoU 0.7186 | val_Dice 0.8340
Epoch 89 | loss 0.5011 | val_OA 0.8509 | val_IoU 0.7191 | val_Dice 0.8343
Epoch 90 | loss 0.5021 | val_OA 0.8553 | val_IoU 0.7304 | val_Dice 0.8425
Epoch 91 | loss 0.4815 | val_OA 0.8506 | val_IoU 0.7199 | val_Dice 0.8349
Epoch 92 | loss 0.4854 | val_OA 0.8545 | val_IoU 0.7324 | val_Dice 0.8441
Epoch 93 | loss 0.4921 | val_OA 0.8503 | val_IoU 0.7238 | val_Dice 0.8381
Epoch 94 | loss 0.4913 | val_OA 0.8486 | val_IoU 0.7149 | val_Dice 0.8313
Epoch 95 | loss 0.5007 | val_OA 0.8541 | val_IoU 0.7289 | val_Dice 0.8414
Epoch 96 | loss 0.4915 | val_OA 0.8498 | val_IoU 0.7186 | val_Dice 0.8340
Epoch 97 | loss 0.4869 | val_OA 0.8551 | val_IoU 0.7334 | val_Dice 0.8448
Epoch 98 | loss 0.4984 | val_OA 0.8467 | val_IoU 0.7177 | val_Dice 0.8339
Epoch 99 | loss 0.5021 | val_OA 0.8546 | val_IoU 0.7330 | val_Dice 0.8446
Epoch 100 | loss 0.4950 | val_OA 0.8550 | val_IoU 0.7308 | val_Dice 0.8428

=== Fraction 30% | train images: 681 ===
Epoch 01 | loss 0.9192 | val_OA 0.7593 | val_IoU 0.5971 | val_Dice 0.7447
Epoch 02 | loss 0.8406 | val_OA 0.7834 | val_IoU 0.6277 | val_Dice 0.7685
Epoch 03 | loss 0.7857 | val_OA 0.7795 | val_IoU 0.6180 | val_Dice 0.7601
Epoch 04 | loss 0.7857 | val_OA 0.8021 | val_IoU 0.6502 | val_Dice 0.7850
Epoch 05 | loss 0.7471 | val_OA 0.7972 | val_IoU 0.6405 | val_Dice 0.7772
Epoch 06 | loss 0.7382 | val_OA 0.8056 | val_IoU 0.6587 | val_Dice 0.7918
Epoch 07 | loss 0.7123 | val_OA 0.8138 | val_IoU 0.6698 | val_Dice 0.7999
Epoch 08 | loss 0.7091 | val_OA 0.8129 | val_IoU 0.6693 | val_Dice 0.7997
Epoch 09 | loss 0.6949 | val_OA 0.8135 | val_IoU 0.6641 | val_Dice 0.7950
Epoch 10 | loss 0.7022 | val_OA 0.8110 | val_IoU 0.6665 | val_Dice 0.7977
Epoch 11 | loss 0.7075 | val_OA 0.8092 | val_IoU 0.6604 | val_Dice 0.7926
Epoch 12 | loss 0.6656 | val_OA 0.8201 | val_IoU 0.6723 | val_Dice 0.8008
Epoch 13 | loss 0.6621 | val_OA 0.8188 | val_IoU 0.6704 | val_Dice 0.7995
Epoch 14 | loss 0.6476 | val_OA 0.8309 | val_IoU 0.6888 | val_Dice 0.8129
Epoch 15 | loss 0.6639 | val_OA 0.8275 | val_IoU 0.6784 | val_Dice 0.8047
Epoch 16 | loss 0.6375 | val_OA 0.8286 | val_IoU 0.6909 | val_Dice 0.8151
Epoch 17 | loss 0.6577 | val_OA 0.8303 | val_IoU 0.6908 | val_Dice 0.8146
Epoch 18 | loss 0.6470 | val_OA 0.8074 | val_IoU 0.6533 | val_Dice 0.7867
Epoch 19 | loss 0.6399 | val_OA 0.8359 | val_IoU 0.7004 | val_Dice 0.8217
Epoch 20 | loss 0.6274 | val_OA 0.8372 | val_IoU 0.7019 | val_Dice 0.8227
Epoch 21 | loss 0.6158 | val_OA 0.8378 | val_IoU 0.6993 | val_Dice 0.8204
Epoch 22 | loss 0.6106 | val_OA 0.8350 | val_IoU 0.6911 | val_Dice 0.8140
Epoch 23 | loss 0.6011 | val_OA 0.8295 | val_IoU 0.6827 | val_Dice 0.8080
Epoch 24 | loss 0.6015 | val_OA 0.8440 | val_IoU 0.7117 | val_Dice 0.8295
Epoch 25 | loss 0.5927 | val_OA 0.8373 | val_IoU 0.7015 | val_Dice 0.8223
Epoch 26 | loss 0.5988 | val_OA 0.8353 | val_IoU 0.7056 | val_Dice 0.8260
Epoch 27 | loss 0.5937 | val_OA 0.8433 | val_IoU 0.7135 | val_Dice 0.8311
Epoch 28 | loss 0.5889 | val_OA 0.8378 | val_IoU 0.7010 | val_Dice 0.8218
Epoch 29 | loss 0.5873 | val_OA 0.8431 | val_IoU 0.7129 | val_Dice 0.8305
```

```
Epoch 30 | loss 0.5917 | val_OA 0.8433 | val_IoU 0.7104 | val_Dice 0.8286
Epoch 31 | loss 0.6062 | val_OA 0.8414 | val_IoU 0.7122 | val_Dice 0.8303
Epoch 32 | loss 0.5833 | val_OA 0.8432 | val_IoU 0.7162 | val_Dice 0.8332
Epoch 33 | loss 0.5675 | val_OA 0.8349 | val_IoU 0.6920 | val_Dice 0.8149
Epoch 34 | loss 0.5851 | val_OA 0.8454 | val_IoU 0.7196 | val_Dice 0.8355
Epoch 35 | loss 0.5757 | val_OA 0.8511 | val_IoU 0.7264 | val_Dice 0.8400
Epoch 36 | loss 0.5548 | val_OA 0.8404 | val_IoU 0.7070 | val_Dice 0.8263
Epoch 37 | loss 0.5673 | val_OA 0.8419 | val_IoU 0.7015 | val_Dice 0.8214
Epoch 38 | loss 0.5647 | val_OA 0.8410 | val_IoU 0.7058 | val_Dice 0.8252
Epoch 39 | loss 0.5761 | val_OA 0.8434 | val_IoU 0.7153 | val_Dice 0.8325
Epoch 40 | loss 0.5763 | val_OA 0.8438 | val_IoU 0.7057 | val_Dice 0.8246
Epoch 41 | loss 0.5709 | val_OA 0.8539 | val_IoU 0.7274 | val_Dice 0.8403
Epoch 42 | loss 0.5583 | val_OA 0.8221 | val_IoU 0.6772 | val_Dice 0.8047
Epoch 43 | loss 0.5740 | val_OA 0.8498 | val_IoU 0.7208 | val_Dice 0.8357
Epoch 44 | loss 0.5642 | val_OA 0.8415 | val_IoU 0.7100 | val_Dice 0.8285
Epoch 45 | loss 0.5577 | val_OA 0.8461 | val_IoU 0.7091 | val_Dice 0.8270
Epoch 46 | loss 0.5494 | val_OA 0.8504 | val_IoU 0.7207 | val_Dice 0.8356
Epoch 47 | loss 0.5499 | val_OA 0.8506 | val_IoU 0.7243 | val_Dice 0.8384
Epoch 48 | loss 0.5547 | val_OA 0.8496 | val_IoU 0.7158 | val_Dice 0.8318
Epoch 49 | loss 0.5503 | val_OA 0.8500 | val_IoU 0.7225 | val_Dice 0.8371
Epoch 50 | loss 0.5511 | val_OA 0.8503 | val_IoU 0.7193 | val_Dice 0.8345
Epoch 51 | loss 0.5333 | val_OA 0.8576 | val_IoU 0.7333 | val_Dice 0.8443
Epoch 52 | loss 0.5393 | val_OA 0.8527 | val_IoU 0.7236 | val_Dice 0.8375
Epoch 53 | loss 0.5484 | val_OA 0.8464 | val_IoU 0.7092 | val_Dice 0.8270
Epoch 54 | loss 0.5314 | val_OA 0.8542 | val_IoU 0.7311 | val_Dice 0.8432
Epoch 55 | loss 0.5482 | val_OA 0.8489 | val_IoU 0.7174 | val_Dice 0.8332
Epoch 56 | loss 0.5514 | val_OA 0.8492 | val_IoU 0.7183 | val_Dice 0.8339
Epoch 57 | loss 0.5249 | val_OA 0.8555 | val_IoU 0.7297 | val_Dice 0.8419
Epoch 58 | loss 0.5281 | val_OA 0.8519 | val_IoU 0.7240 | val_Dice 0.8380
Epoch 59 | loss 0.5292 | val_OA 0.8517 | val_IoU 0.7186 | val_Dice 0.8337
Epoch 60 | loss 0.5258 | val_OA 0.8591 | val_IoU 0.7380 | val_Dice 0.8478
Epoch 61 | loss 0.5194 | val_OA 0.8462 | val_IoU 0.7097 | val_Dice 0.8274
Epoch 62 | loss 0.5323 | val_OA 0.8616 | val_IoU 0.7420 | val_Dice 0.8505
Epoch 63 | loss 0.5355 | val_OA 0.8558 | val_IoU 0.7306 | val_Dice 0.8425
Epoch 64 | loss 0.5292 | val_OA 0.8590 | val_IoU 0.7355 | val_Dice 0.8458
Epoch 65 | loss 0.5299 | val_OA 0.8552 | val_IoU 0.7311 | val_Dice 0.8430
Epoch 66 | loss 0.5238 | val_OA 0.8517 | val_IoU 0.7197 | val_Dice 0.8346
Epoch 67 | loss 0.5164 | val_OA 0.8561 | val_IoU 0.7282 | val_Dice 0.8406
Epoch 68 | loss 0.5255 | val_OA 0.8600 | val_IoU 0.7412 | val_Dice 0.8501
Epoch 69 | loss 0.5142 | val_OA 0.8593 | val_IoU 0.7388 | val_Dice 0.8483
Epoch 70 | loss 0.5231 | val_OA 0.8642 | val_IoU 0.7455 | val_Dice 0.8528
Epoch 71 | loss 0.5123 | val_OA 0.8603 | val_IoU 0.7379 | val_Dice 0.8475
Epoch 72 | loss 0.5147 | val_OA 0.8629 | val_IoU 0.7414 | val_Dice 0.8498
Epoch 73 | loss 0.5032 | val_OA 0.8542 | val_IoU 0.7242 | val_Dice 0.8377
Epoch 74 | loss 0.5191 | val_OA 0.8623 | val_IoU 0.7427 | val_Dice 0.8509
Epoch 75 | loss 0.5287 | val_OA 0.8503 | val_IoU 0.7149 | val_Dice 0.8310
Epoch 76 | loss 0.5173 | val_OA 0.8582 | val_IoU 0.7329 | val_Dice 0.8439
Epoch 77 | loss 0.5255 | val_OA 0.8562 | val_IoU 0.7322 | val_Dice 0.8437
Epoch 78 | loss 0.5118 | val_OA 0.8603 | val_IoU 0.7395 | val_Dice 0.8487
Epoch 79 | loss 0.5335 | val_OA 0.8599 | val_IoU 0.7414 | val_Dice 0.8503
Epoch 80 | loss 0.5137 | val_OA 0.8638 | val_IoU 0.7448 | val_Dice 0.8523
Epoch 81 | loss 0.5094 | val_OA 0.8496 | val_IoU 0.7133 | val_Dice 0.8298
Epoch 82 | loss 0.5143 | val_OA 0.8619 | val_IoU 0.7444 | val_Dice 0.8522
Epoch 83 | loss 0.5146 | val_OA 0.8637 | val_IoU 0.7459 | val_Dice 0.8531
Epoch 84 | loss 0.4960 | val_OA 0.8626 | val_IoU 0.7407 | val_Dice 0.8493
Epoch 85 | loss 0.4910 | val_OA 0.8560 | val_IoU 0.7303 | val_Dice 0.8423
Epoch 86 | loss 0.5003 | val_OA 0.8627 | val_IoU 0.7420 | val_Dice 0.8502
Epoch 87 | loss 0.4959 | val_OA 0.8640 | val_IoU 0.7456 | val_Dice 0.8528
Epoch 88 | loss 0.5049 | val_OA 0.8636 | val_IoU 0.7484 | val_Dice 0.8550
Epoch 89 | loss 0.5039 | val_OA 0.8646 | val_IoU 0.7475 | val_Dice 0.8542
```

```
Epoch 90 | loss 0.4888 | val_OA 0.8571 | val_IoU 0.7333 | val_Dice 0.8445
Epoch 91 | loss 0.4905 | val_OA 0.8640 | val_IoU 0.7453 | val_Dice 0.8526
Epoch 92 | loss 0.5056 | val_OA 0.8657 | val_IoU 0.7478 | val_Dice 0.8543
Epoch 93 | loss 0.4779 | val_OA 0.8579 | val_IoU 0.7297 | val_Dice 0.8415
Epoch 94 | loss 0.5014 | val_OA 0.8503 | val_IoU 0.7308 | val_Dice 0.8435
Epoch 95 | loss 0.5038 | val_OA 0.8643 | val_IoU 0.7476 | val_Dice 0.8543
Epoch 96 | loss 0.4844 | val_OA 0.8563 | val_IoU 0.7293 | val_Dice 0.8414
Epoch 97 | loss 0.4898 | val_OA 0.8569 | val_IoU 0.7343 | val_Dice 0.8452
Epoch 98 | loss 0.5039 | val_OA 0.8577 | val_IoU 0.7318 | val_Dice 0.8432
Epoch 99 | loss 0.4920 | val_OA 0.8612 | val_IoU 0.7448 | val_Dice 0.8526
Epoch 100 | loss 0.5078 | val_OA 0.8659 | val_IoU 0.7505 | val_Dice 0.8562

=== Fraction 50% | train images: 1136 ===
Epoch 01 | loss 0.8842 | val_OA 0.7675 | val_IoU 0.5963 | val_Dice 0.7418
Epoch 02 | loss 0.7926 | val_OA 0.7948 | val_IoU 0.6440 | val_Dice 0.7810
Epoch 03 | loss 0.7590 | val_OA 0.7879 | val_IoU 0.6309 | val_Dice 0.7704
Epoch 04 | loss 0.7406 | val_OA 0.7924 | val_IoU 0.6347 | val_Dice 0.7729
Epoch 05 | loss 0.7182 | val_OA 0.8098 | val_IoU 0.6661 | val_Dice 0.7975
Epoch 06 | loss 0.7161 | val_OA 0.8149 | val_IoU 0.6706 | val_Dice 0.8004
Epoch 07 | loss 0.6851 | val_OA 0.8217 | val_IoU 0.6806 | val_Dice 0.8077
Epoch 08 | loss 0.6862 | val_OA 0.8180 | val_IoU 0.6754 | val_Dice 0.8039
Epoch 09 | loss 0.6741 | val_OA 0.8147 | val_IoU 0.6593 | val_Dice 0.7905
Epoch 10 | loss 0.6667 | val_OA 0.8215 | val_IoU 0.6807 | val_Dice 0.8078
Epoch 11 | loss 0.6529 | val_OA 0.8217 | val_IoU 0.6656 | val_Dice 0.7946
Epoch 12 | loss 0.6419 | val_OA 0.8279 | val_IoU 0.6958 | val_Dice 0.8193
Epoch 13 | loss 0.6537 | val_OA 0.8214 | val_IoU 0.6733 | val_Dice 0.8014
Epoch 14 | loss 0.6329 | val_OA 0.8299 | val_IoU 0.6846 | val_Dice 0.8095
Epoch 15 | loss 0.6272 | val_OA 0.8439 | val_IoU 0.7115 | val_Dice 0.8293
Epoch 16 | loss 0.6272 | val_OA 0.8064 | val_IoU 0.6436 | val_Dice 0.7780
Epoch 17 | loss 0.6277 | val_OA 0.8346 | val_IoU 0.6932 | val_Dice 0.8159
Epoch 18 | loss 0.6106 | val_OA 0.8476 | val_IoU 0.7171 | val_Dice 0.8332
Epoch 19 | loss 0.6014 | val_OA 0.8391 | val_IoU 0.7043 | val_Dice 0.8243
Epoch 20 | loss 0.6184 | val_OA 0.8402 | val_IoU 0.7089 | val_Dice 0.8278
Epoch 21 | loss 0.5929 | val_OA 0.8309 | val_IoU 0.6849 | val_Dice 0.8096
Epoch 22 | loss 0.6070 | val_OA 0.8471 | val_IoU 0.7162 | val_Dice 0.8326
Epoch 23 | loss 0.5849 | val_OA 0.8370 | val_IoU 0.6998 | val_Dice 0.8210
Epoch 24 | loss 0.5974 | val_OA 0.8461 | val_IoU 0.7193 | val_Dice 0.8352
Epoch 25 | loss 0.5801 | val_OA 0.8509 | val_IoU 0.7252 | val_Dice 0.8390
Epoch 26 | loss 0.5821 | val_OA 0.8456 | val_IoU 0.7102 | val_Dice 0.8280
Epoch 27 | loss 0.5733 | val_OA 0.8454 | val_IoU 0.7165 | val_Dice 0.8331
Epoch 28 | loss 0.5765 | val_OA 0.8530 | val_IoU 0.7270 | val_Dice 0.8402
Epoch 29 | loss 0.5711 | val_OA 0.8525 | val_IoU 0.7253 | val_Dice 0.8389
Epoch 30 | loss 0.5665 | val_OA 0.8583 | val_IoU 0.7361 | val_Dice 0.8464
Epoch 31 | loss 0.5634 | val_OA 0.8564 | val_IoU 0.7316 | val_Dice 0.8432
Epoch 32 | loss 0.5713 | val_OA 0.8536 | val_IoU 0.7279 | val_Dice 0.8408
Epoch 33 | loss 0.5580 | val_OA 0.8530 | val_IoU 0.7301 | val_Dice 0.8426
Epoch 34 | loss 0.5602 | val_OA 0.8479 | val_IoU 0.7132 | val_Dice 0.8300
Epoch 35 | loss 0.5588 | val_OA 0.8563 | val_IoU 0.7303 | val_Dice 0.8422
Epoch 36 | loss 0.5623 | val_OA 0.8599 | val_IoU 0.7400 | val_Dice 0.8491
Epoch 37 | loss 0.5626 | val_OA 0.8548 | val_IoU 0.7349 | val_Dice 0.8460
Epoch 38 | loss 0.5588 | val_OA 0.8587 | val_IoU 0.7364 | val_Dice 0.8465
Epoch 39 | loss 0.5476 | val_OA 0.8580 | val_IoU 0.7339 | val_Dice 0.8448
Epoch 40 | loss 0.5514 | val_OA 0.8567 | val_IoU 0.7299 | val_Dice 0.8418
Epoch 41 | loss 0.5422 | val_OA 0.8584 | val_IoU 0.7315 | val_Dice 0.8428
Epoch 42 | loss 0.5515 | val_OA 0.8651 | val_IoU 0.7465 | val_Dice 0.8533
Epoch 43 | loss 0.5485 | val_OA 0.8603 | val_IoU 0.7378 | val_Dice 0.8474
Epoch 44 | loss 0.5449 | val_OA 0.8635 | val_IoU 0.7436 | val_Dice 0.8514
Epoch 45 | loss 0.5382 | val_OA 0.8598 | val_IoU 0.7385 | val_Dice 0.8480
Epoch 46 | loss 0.5420 | val_OA 0.8541 | val_IoU 0.7291 | val_Dice 0.8417
Epoch 47 | loss 0.5451 | val_OA 0.8550 | val_IoU 0.7283 | val_Dice 0.8408
```

```
Epoch 48 | loss 0.5498 | val_OA 0.8412 | val_IoU 0.6981 | val_Dice 0.8188
Epoch 49 | loss 0.5436 | val_OA 0.8606 | val_IoU 0.7424 | val_Dice 0.8509
Epoch 50 | loss 0.5336 | val_OA 0.8564 | val_IoU 0.7271 | val_Dice 0.8396
Epoch 51 | loss 0.5383 | val_OA 0.8664 | val_IoU 0.7483 | val_Dice 0.8546
Epoch 52 | loss 0.5315 | val_OA 0.8603 | val_IoU 0.7397 | val_Dice 0.8488
Epoch 53 | loss 0.5246 | val_OA 0.8607 | val_IoU 0.7426 | val_Dice 0.8510
Epoch 54 | loss 0.5271 | val_OA 0.8676 | val_IoU 0.7506 | val_Dice 0.8561
Epoch 55 | loss 0.5276 | val_OA 0.8595 | val_IoU 0.7361 | val_Dice 0.8462
Epoch 56 | loss 0.5297 | val_OA 0.8640 | val_IoU 0.7472 | val_Dice 0.8540
Epoch 57 | loss 0.5169 | val_OA 0.8668 | val_IoU 0.7500 | val_Dice 0.8558
Epoch 58 | loss 0.5283 | val_OA 0.8591 | val_IoU 0.7380 | val_Dice 0.8478
Epoch 59 | loss 0.5232 | val_OA 0.8660 | val_IoU 0.7461 | val_Dice 0.8529
Epoch 60 | loss 0.5145 | val_OA 0.8635 | val_IoU 0.7451 | val_Dice 0.8526
Epoch 61 | loss 0.5338 | val_OA 0.8657 | val_IoU 0.7463 | val_Dice 0.8531
Epoch 62 | loss 0.5204 | val_OA 0.8631 | val_IoU 0.7429 | val_Dice 0.8509
Epoch 63 | loss 0.5321 | val_OA 0.8648 | val_IoU 0.7454 | val_Dice 0.8526
Epoch 64 | loss 0.5154 | val_OA 0.8672 | val_IoU 0.7484 | val_Dice 0.8545
Epoch 65 | loss 0.5116 | val_OA 0.8607 | val_IoU 0.7400 | val_Dice 0.8490
Epoch 66 | loss 0.5265 | val_OA 0.8673 | val_IoU 0.7534 | val_Dice 0.8582
Epoch 67 | loss 0.5136 | val_OA 0.8674 | val_IoU 0.7513 | val_Dice 0.8566
Epoch 68 | loss 0.5196 | val_OA 0.8564 | val_IoU 0.7315 | val_Dice 0.8431
Epoch 69 | loss 0.5099 | val_OA 0.8693 | val_IoU 0.7535 | val_Dice 0.8580
Epoch 70 | loss 0.5104 | val_OA 0.8660 | val_IoU 0.7442 | val_Dice 0.8514
Epoch 71 | loss 0.5198 | val_OA 0.8678 | val_IoU 0.7502 | val_Dice 0.8557
Epoch 72 | loss 0.5042 | val_OA 0.8672 | val_IoU 0.7502 | val_Dice 0.8559
Epoch 73 | loss 0.5042 | val_OA 0.8673 | val_IoU 0.7519 | val_Dice 0.8571
Epoch 74 | loss 0.5100 | val_OA 0.8698 | val_IoU 0.7561 | val_Dice 0.8599
Epoch 75 | loss 0.5072 | val_OA 0.8583 | val_IoU 0.7290 | val_Dice 0.8409
Epoch 76 | loss 0.5117 | val_OA 0.8636 | val_IoU 0.7464 | val_Dice 0.8535
Epoch 77 | loss 0.5085 | val_OA 0.8608 | val_IoU 0.7356 | val_Dice 0.8456
Epoch 78 | loss 0.4973 | val_OA 0.8639 | val_IoU 0.7476 | val_Dice 0.8544
Epoch 79 | loss 0.5148 | val_OA 0.8616 | val_IoU 0.7389 | val_Dice 0.8481
Epoch 80 | loss 0.5060 | val_OA 0.8721 | val_IoU 0.7593 | val_Dice 0.8620
Epoch 81 | loss 0.4988 | val_OA 0.8706 | val_IoU 0.7551 | val_Dice 0.8590
Epoch 82 | loss 0.5096 | val_OA 0.8708 | val_IoU 0.7570 | val_Dice 0.8604
Epoch 83 | loss 0.4974 | val_OA 0.8627 | val_IoU 0.7430 | val_Dice 0.8510
Epoch 84 | loss 0.4973 | val_OA 0.8696 | val_IoU 0.7536 | val_Dice 0.8580
Epoch 85 | loss 0.5018 | val_OA 0.8701 | val_IoU 0.7532 | val_Dice 0.8577
Epoch 86 | loss 0.4954 | val_OA 0.8702 | val_IoU 0.7552 | val_Dice 0.8592
Epoch 87 | loss 0.5039 | val_OA 0.8630 | val_IoU 0.7409 | val_Dice 0.8494
Epoch 88 | loss 0.5005 | val_OA 0.8706 | val_IoU 0.7569 | val_Dice 0.8604
Epoch 89 | loss 0.5085 | val_OA 0.8721 | val_IoU 0.7581 | val_Dice 0.8610
Epoch 90 | loss 0.4968 | val_OA 0.8658 | val_IoU 0.7498 | val_Dice 0.8557
Epoch 91 | loss 0.4906 | val_OA 0.8665 | val_IoU 0.7462 | val_Dice 0.8529
Epoch 92 | loss 0.4900 | val_OA 0.8713 | val_IoU 0.7583 | val_Dice 0.8613
Epoch 93 | loss 0.4928 | val_OA 0.8719 | val_IoU 0.7572 | val_Dice 0.8604
Epoch 94 | loss 0.4894 | val_OA 0.8630 | val_IoU 0.7429 | val_Dice 0.8509
Epoch 95 | loss 0.4941 | val_OA 0.8713 | val_IoU 0.7560 | val_Dice 0.8596
Epoch 96 | loss 0.4935 | val_OA 0.8667 | val_IoU 0.7468 | val_Dice 0.8533
Epoch 97 | loss 0.4938 | val_OA 0.8684 | val_IoU 0.7518 | val_Dice 0.8569
Epoch 98 | loss 0.4908 | val_OA 0.8728 | val_IoU 0.7592 | val_Dice 0.8618
Epoch 99 | loss 0.4863 | val_OA 0.8731 | val_IoU 0.7618 | val_Dice 0.8637
Epoch 100 | loss 0.4874 | val_OA 0.8681 | val_IoU 0.7516 | val_Dice 0.8567

=== Fraction 100% | train images: 2273 ===
Epoch 01 | loss 0.8621 | val_OA 0.7885 | val_IoU 0.6356 | val_Dice 0.7747
Epoch 02 | loss 0.7567 | val_OA 0.7998 | val_IoU 0.6482 | val_Dice 0.7837
Epoch 03 | loss 0.7354 | val_OA 0.7899 | val_IoU 0.6127 | val_Dice 0.7524
Epoch 04 | loss 0.7136 | val_OA 0.7970 | val_IoU 0.6374 | val_Dice 0.7744
Epoch 05 | loss 0.7004 | val_OA 0.8208 | val_IoU 0.6729 | val_Dice 0.8012
```

```
Epoch 06 | loss 0.6565 | val_OA 0.8025 | val_IoU 0.6456 | val_Dice 0.7807
Epoch 07 | loss 0.6584 | val_OA 0.8195 | val_IoU 0.6617 | val_Dice 0.7916
Epoch 08 | loss 0.6328 | val_OA 0.8088 | val_IoU 0.6459 | val_Dice 0.7795
Epoch 09 | loss 0.6301 | val_OA 0.8412 | val_IoU 0.7119 | val_Dice 0.8301
Epoch 10 | loss 0.6213 | val_OA 0.8275 | val_IoU 0.6895 | val_Dice 0.8141
Epoch 11 | loss 0.6148 | val_OA 0.8306 | val_IoU 0.6992 | val_Dice 0.8216
Epoch 12 | loss 0.6193 | val_OA 0.8440 | val_IoU 0.7088 | val_Dice 0.8271
Epoch 13 | loss 0.6090 | val_OA 0.8405 | val_IoU 0.7031 | val_Dice 0.8231
Epoch 14 | loss 0.5861 | val_OA 0.8509 | val_IoU 0.7248 | val_Dice 0.8388
Epoch 15 | loss 0.6092 | val_OA 0.8545 | val_IoU 0.7302 | val_Dice 0.8424
Epoch 16 | loss 0.5918 | val_OA 0.8494 | val_IoU 0.7202 | val_Dice 0.8354
Epoch 17 | loss 0.5783 | val_OA 0.8477 | val_IoU 0.7185 | val_Dice 0.8343
Epoch 18 | loss 0.5817 | val_OA 0.8450 | val_IoU 0.7178 | val_Dice 0.8342
Epoch 19 | loss 0.5658 | val_OA 0.8549 | val_IoU 0.7292 | val_Dice 0.8416
Epoch 20 | loss 0.5739 | val_OA 0.8473 | val_IoU 0.7234 | val_Dice 0.8382
Epoch 21 | loss 0.5922 | val_OA 0.8575 | val_IoU 0.7343 | val_Dice 0.8452
Epoch 22 | loss 0.5646 | val_OA 0.8582 | val_IoU 0.7347 | val_Dice 0.8453
Epoch 23 | loss 0.5550 | val_OA 0.8531 | val_IoU 0.7292 | val_Dice 0.8419
Epoch 24 | loss 0.5530 | val_OA 0.8525 | val_IoU 0.7275 | val_Dice 0.8406
Epoch 25 | loss 0.5482 | val_OA 0.8633 | val_IoU 0.7429 | val_Dice 0.8508
Epoch 26 | loss 0.5575 | val_OA 0.8613 | val_IoU 0.7430 | val_Dice 0.8513
Epoch 27 | loss 0.6018 | val_OA 0.8570 | val_IoU 0.7339 | val_Dice 0.8449
Epoch 28 | loss 0.5576 | val_OA 0.8643 | val_IoU 0.7450 | val_Dice 0.8523
Epoch 29 | loss 0.5499 | val_OA 0.8530 | val_IoU 0.7311 | val_Dice 0.8434
Epoch 30 | loss 0.5440 | val_OA 0.8619 | val_IoU 0.7422 | val_Dice 0.8505
Epoch 31 | loss 0.5501 | val_OA 0.8655 | val_IoU 0.7496 | val_Dice 0.8557
Epoch 32 | loss 0.5491 | val_OA 0.8613 | val_IoU 0.7390 | val_Dice 0.8482
Epoch 33 | loss 0.5379 | val_OA 0.8545 | val_IoU 0.7295 | val_Dice 0.8419
Epoch 34 | loss 0.5379 | val_OA 0.8618 | val_IoU 0.7434 | val_Dice 0.8515
Epoch 35 | loss 0.5384 | val_OA 0.8632 | val_IoU 0.7465 | val_Dice 0.8536
Epoch 36 | loss 0.5221 | val_OA 0.8579 | val_IoU 0.7392 | val_Dice 0.8489
Epoch 37 | loss 0.5337 | val_OA 0.8655 | val_IoU 0.7491 | val_Dice 0.8553
Epoch 38 | loss 0.5240 | val_OA 0.8651 | val_IoU 0.7477 | val_Dice 0.8542
Epoch 39 | loss 0.5352 | val_OA 0.8676 | val_IoU 0.7516 | val_Dice 0.8568
Epoch 40 | loss 0.5136 | val_OA 0.8699 | val_IoU 0.7559 | val_Dice 0.8597
Epoch 41 | loss 0.5230 | val_OA 0.8637 | val_IoU 0.7490 | val_Dice 0.8554
Epoch 42 | loss 0.5137 | val_OA 0.8607 | val_IoU 0.7422 | val_Dice 0.8508
Epoch 43 | loss 0.5274 | val_OA 0.8696 | val_IoU 0.7550 | val_Dice 0.8591
Epoch 44 | loss 0.5164 | val_OA 0.8657 | val_IoU 0.7476 | val_Dice 0.8541
Epoch 45 | loss 0.5202 | val_OA 0.8666 | val_IoU 0.7535 | val_Dice 0.8584
Epoch 46 | loss 0.5143 | val_OA 0.8688 | val_IoU 0.7506 | val_Dice 0.8559
Epoch 47 | loss 0.5173 | val_OA 0.8647 | val_IoU 0.7469 | val_Dice 0.8537
Epoch 48 | loss 0.5161 | val_OA 0.8720 | val_IoU 0.7607 | val_Dice 0.8630
Epoch 49 | loss 0.5161 | val_OA 0.8693 | val_IoU 0.7539 | val_Dice 0.8583
Epoch 50 | loss 0.5100 | val_OA 0.8641 | val_IoU 0.7440 | val_Dice 0.8516
Epoch 51 | loss 0.5084 | val_OA 0.8710 | val_IoU 0.7578 | val_Dice 0.8610
Epoch 52 | loss 0.5058 | val_OA 0.8680 | val_IoU 0.7522 | val_Dice 0.8572
Epoch 53 | loss 0.5044 | val_OA 0.8738 | val_IoU 0.7620 | val_Dice 0.8637
Epoch 54 | loss 0.5033 | val_OA 0.8683 | val_IoU 0.7553 | val_Dice 0.8595
Epoch 55 | loss 0.5154 | val_OA 0.8726 | val_IoU 0.7587 | val_Dice 0.8615
Epoch 56 | loss 0.5058 | val_OA 0.8675 | val_IoU 0.7529 | val_Dice 0.8579
Epoch 57 | loss 0.5036 | val_OA 0.8674 | val_IoU 0.7546 | val_Dice 0.8591
Epoch 58 | loss 0.4960 | val_OA 0.8695 | val_IoU 0.7557 | val_Dice 0.8596
Epoch 59 | loss 0.5008 | val_OA 0.8707 | val_IoU 0.7570 | val_Dice 0.8604
Epoch 60 | loss 0.4969 | val_OA 0.8713 | val_IoU 0.7579 | val_Dice 0.8610
Epoch 61 | loss 0.4987 | val_OA 0.8718 | val_IoU 0.7573 | val_Dice 0.8605
Epoch 62 | loss 0.4940 | val_OA 0.8652 | val_IoU 0.7436 | val_Dice 0.8511
Epoch 63 | loss 0.5021 | val_OA 0.8735 | val_IoU 0.7629 | val_Dice 0.8644
Epoch 64 | loss 0.4912 | val_OA 0.8643 | val_IoU 0.7429 | val_Dice 0.8507
Epoch 65 | loss 0.4980 | val_OA 0.8731 | val_IoU 0.7588 | val_Dice 0.8614
```

```
Epoch 66 | loss 0.4979 | val_OA 0.8667 | val_IoU 0.7487 | val_Dice 0.8548
Epoch 67 | loss 0.4963 | val_OA 0.8675 | val_IoU 0.7559 | val_Dice 0.8601
Epoch 68 | loss 0.5054 | val_OA 0.8748 | val_IoU 0.7642 | val_Dice 0.8652
Epoch 69 | loss 0.5006 | val_OA 0.8723 | val_IoU 0.7597 | val_Dice 0.8622
Epoch 70 | loss 0.4882 | val_OA 0.8733 | val_IoU 0.7616 | val_Dice 0.8635
Epoch 71 | loss 0.4902 | val_OA 0.8673 | val_IoU 0.7507 | val_Dice 0.8562
Epoch 72 | loss 0.4878 | val_OA 0.8750 | val_IoU 0.7623 | val_Dice 0.8638
Epoch 73 | loss 0.4891 | val_OA 0.8762 | val_IoU 0.7662 | val_Dice 0.8664
Epoch 74 | loss 0.4888 | val_OA 0.8781 | val_IoU 0.7701 | val_Dice 0.8691
Epoch 75 | loss 0.4822 | val_OA 0.8749 | val_IoU 0.7641 | val_Dice 0.8651
Epoch 76 | loss 0.4850 | val_OA 0.8695 | val_IoU 0.7568 | val_Dice 0.8605
Epoch 77 | loss 0.4949 | val_OA 0.8730 | val_IoU 0.7593 | val_Dice 0.8618
Epoch 78 | loss 0.4942 | val_OA 0.8684 | val_IoU 0.7563 | val_Dice 0.8602
Epoch 79 | loss 0.4812 | val_OA 0.8794 | val_IoU 0.7726 | val_Dice 0.8708
Epoch 80 | loss 0.4877 | val_OA 0.8690 | val_IoU 0.7494 | val_Dice 0.8550
Epoch 81 | loss 0.4813 | val_OA 0.8721 | val_IoU 0.7612 | val_Dice 0.8634
Epoch 82 | loss 0.4796 | val_OA 0.8674 | val_IoU 0.7536 | val_Dice 0.8584
Epoch 83 | loss 0.4762 | val_OA 0.8706 | val_IoU 0.7575 | val_Dice 0.8608
Epoch 84 | loss 0.4783 | val_OA 0.8736 | val_IoU 0.7594 | val_Dice 0.8618
Epoch 85 | loss 0.4790 | val_OA 0.8804 | val_IoU 0.7715 | val_Dice 0.8698
Epoch 86 | loss 0.4782 | val_OA 0.8760 | val_IoU 0.7657 | val_Dice 0.8661
Epoch 87 | loss 0.4793 | val_OA 0.8726 | val_IoU 0.7616 | val_Dice 0.8636
Epoch 88 | loss 0.4794 | val_OA 0.8767 | val_IoU 0.7677 | val_Dice 0.8675
Epoch 89 | loss 0.4738 | val_OA 0.8771 | val_IoU 0.7649 | val_Dice 0.8654
Epoch 90 | loss 0.4718 | val_OA 0.8673 | val_IoU 0.7537 | val_Dice 0.8585
Epoch 91 | loss 0.4793 | val_OA 0.8765 | val_IoU 0.7677 | val_Dice 0.8676
Epoch 92 | loss 0.4836 | val_OA 0.8707 | val_IoU 0.7516 | val_Dice 0.8564
Epoch 93 | loss 0.4802 | val_OA 0.8720 | val_IoU 0.7599 | val_Dice 0.8624
Epoch 94 | loss 0.4787 | val_OA 0.8739 | val_IoU 0.7634 | val_Dice 0.8648
Epoch 95 | loss 0.4702 | val_OA 0.8721 | val_IoU 0.7615 | val_Dice 0.8636
Epoch 96 | loss 0.4727 | val_OA 0.8774 | val_IoU 0.7693 | val_Dice 0.8685
Epoch 97 | loss 0.4761 | val_OA 0.8766 | val_IoU 0.7689 | val_Dice 0.8684
Epoch 98 | loss 0.4642 | val_OA 0.8780 | val_IoU 0.7691 | val_Dice 0.8684
Epoch 99 | loss 0.4733 | val_OA 0.8737 | val_IoU 0.7628 | val_Dice 0.8643
Epoch 100 | loss 0.4676 | val_OA 0.8741 | val_IoU 0.7637 | val_Dice 0.8650
```

In [21]:

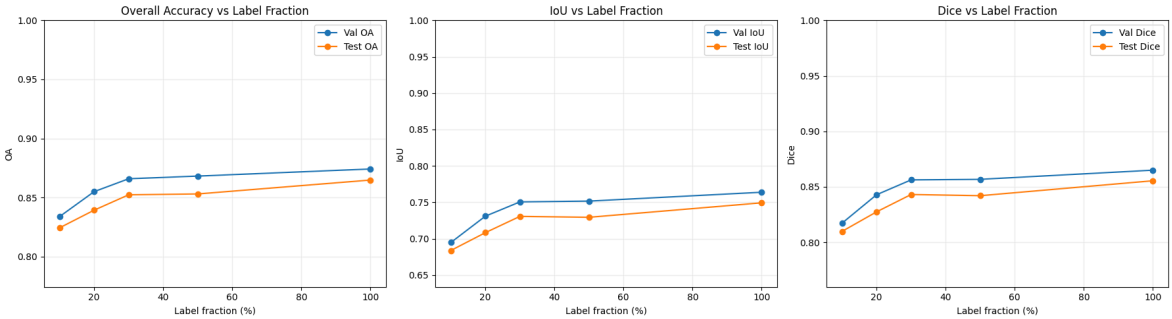
```python
# ----------------------- Results dataframe -----------------------
import pandas as pd
df = pd.DataFrame(RESULTS).sort_values("fraction_pct").reset_index(drop=True)
from pprint import pprint
print("\n== Summary ==")
#pprint(df.to_dict(orient="list"))
df
```

```
== Summary ==
```

Out[21]:

	fraction_pct	n_train_imgs	params_total	params_trainable	model_size_MB	train_time
0	10	227	3319874	3319874	13.279496	795.4602
1	20	454	3319874	3319874	13.279496	1107.4256
2	30	681	3319874	3319874	13.279496	1426.5525
3	50	1136	3319874	3319874	13.279496	2047.1846
4	100	2273	3319874	3319874	13.279496	3601.2109

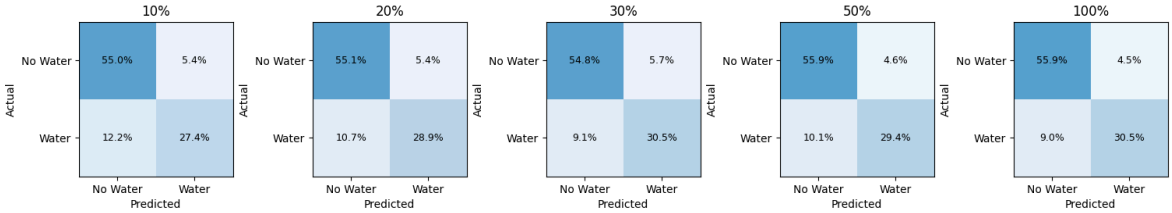
In [22]:
```
# ---------------------------- Plots ------------------------------
plt.figure(figsize=(18,5))
ax1=plt.subplot(1,3,1)
ax1.plot(df["fraction_pct"], df["val_OA"], 'o-', label="Val OA")
ax1.plot(df["fraction_pct"], df["test_OA"], 'o-', label="Test OA")
ax1.set_title("Overall Accuracy vs Label Fraction"); ax1.set_xlabel("Label fract
ax1.set_ylim(max(0.5, df[["val_OA","test_OA"]].min().min()-0.05), 1.0)
ax1.grid(True,alpha=0.2); ax1.legend()

ax2=plt.subplot(1,3,2)
ax2.plot(df["fraction_pct"], df["val_IoU"], 'o-', label="Val IoU")
ax2.plot(df["fraction_pct"], df["test_IoU"], 'o-', label="Test IoU")
ax2.set_title("IoU vs Label Fraction"); ax2.set_xlabel("Label fraction (%)"); ax
ax2.set_ylim(max(0.3, df[["val_IoU","test_IoU"]].min().min()-0.05), 1.0)
ax2.grid(True,alpha=0.2); ax2.legend()

ax3=plt.subplot(1,3,3)
ax3.plot(df["fraction_pct"], df["val_Dice"], 'o-', label="Val Dice")
ax3.plot(df["fraction_pct"], df["test_Dice"], 'o-', label="Test Dice")
ax3.set_title("Dice vs Label Fraction"); ax3.set_xlabel("Label fraction (%)"); a
ax3.set_ylim(max(0.3, df[["val_Dice","test_Dice"]].min().min()-0.05), 1.0)
ax3.grid(True,alpha=0.2); ax3.legend()
plt.tight_layout(); plt.show()
```



In [29]:
```
# ================= Confusion matrices per fraction (binary) =================
import numpy as np
import matplotlib.pyplot as plt
import torch

# ---- robust loader that works with your saved checkpoints ----
def load_ckpt_any(path, device="cpu", default_in_ch=768, default_num_classes=2,
    import torch, numpy as np, torch.serialization as ts
    try:
        ck = torch.load(path, map_location=device, weights_only=True)
    except Exception:
        try:
            from numpy.core.multiarray import _reconstruct
            ts.add_safe_globals([_reconstruct, np.dtype])
            ck = torch.load(path, map_location=device, weights_only=True)
        except Exception:
            print("[warn] weights_only=False fallback (ensure you trust this fil
            ck = torch.load(path, map_location=device, weights_only=False)

    if isinstance(ck, dict) and "decoder_state" in ck:
        state = ck["decoder_state"]
        meta  = {
            "in_ch":       ck.get("in_ch", default_in_ch),
            "num_classes":ck.get("num_classes", default_num_classes),
            "img_size":    ck.get("img_size", default_img_size),
```

```
        }
    elif isinstance(ck, dict):
        state = ck
        meta  = {"in_ch": default_in_ch, "num_classes": default_num_classes, "im
    else:
        raise RuntimeError(f"Unsupported checkpoint type: {type(ck)}")
    return state, meta

@torch.no_grad()
def confusion_on_loader(decoder, loader):
    """Return TN, FP, FN, TP over the whole loader (binary segmentation)."""
    decoder.eval()
    TN = FP = FN = TP = 0
    for b in loader:
        x = b["image"].to(device)
        y = b["mask"].to(device)                # [B,H,W], values in {0,1}
        f = forward_feats(x)
        pred = decoder(f).argmax(1)             # [B,H,W] in {0,1}

        # move to CPU numpy to count
        p = pred.cpu().numpy().ravel()
        g = y.cpu().numpy().ravel()
        TN += int(np.sum((p==0) & (g==0)))
        FP += int(np.sum((p==1) & (g==0)))
        FN += int(np.sum((p==0) & (g==1)))
        TP += int(np.sum((p==1) & (g==1)))
    return TN, FP, FN, TP

def derive_metrics(TN, FP, FN, TP):
    tot = TN + FP + FN + TP
    oa   = (TP + TN) / max(1, tot)
    prec = TP / max(1, (TP + FP))
    rec  = TP / max(1, (TP + FN))
    f1   = 2*prec*rec / max(1e-9, (prec + rec))
    iou_water = TP / max(1, (TP + FP + FN))
    iou_bg    = TN / max(1, (TN + FN + FP))
    miou = 0.5 * (iou_water + iou_bg)
    return dict(OA=oa, Precision=prec, Recall=rec, F1=f1, IoU_water=iou_water, I

def plot_cm(ax, TN, FP, FN, TP, title=""):
    import numpy as np
    cm = np.array([[TN, FP],[FN, TP]], dtype=float)
    cmn = cm / max(1, cm.sum())
    im = ax.imshow(cmn, vmin=0, vmax=1, cmap="Blues")
    for (i,j),v in np.ndenumerate(cmn):
        ax.text(j, i, f"{v*100:.1f}%", ha="center", va="center", fontsize=9)
    ax.set_xticks([0,1]); ax.set_yticks([0,1])
    ax.set_xticklabels(["No Water","Water"])
    ax.set_yticklabels(["No Water","Water"])
    ax.set_xlabel("Predicted"); ax.set_ylabel("Actual")
    ax.set_title(title)
    return im

# ------------------ run for every fraction in your df ------------------
# Ensure we have a list of test pairs if we want to print one example later (opt
try:
    test_ds = test_dl.dataset
except NameError:
    raise RuntimeError("test_dl (or test_ds) not found. Please run your dataload
```

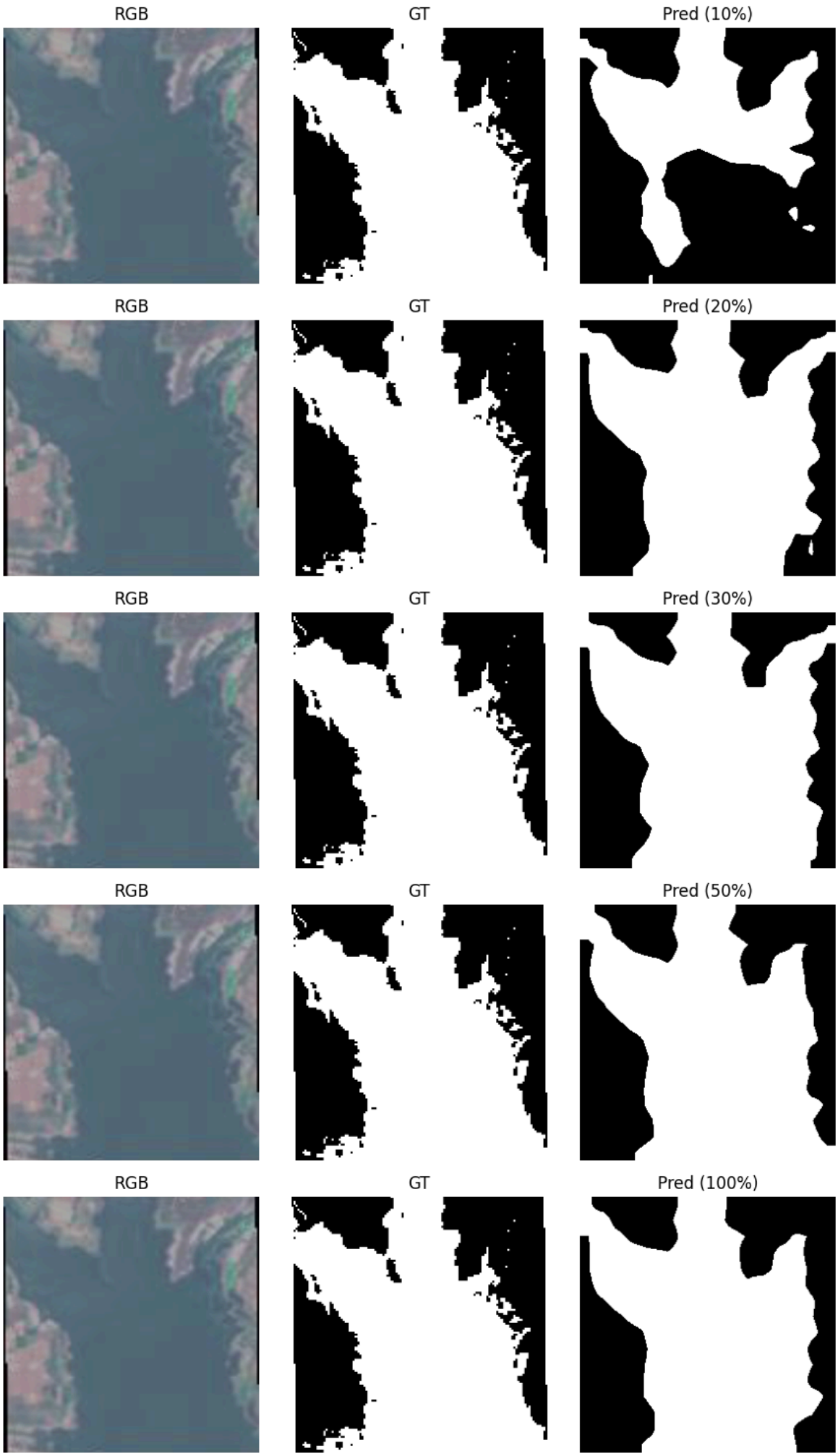
```
# ------------------ plot a CM per fraction ------------------
n = len(cm_df)
fig, axes = plt.subplots(1, n, figsize=(3*n, 3.8), constrained_layout=True)
if n == 1: axes = [axes]
for ax, r, cm_vals in zip(axes, cm_df.itertuples(index=False), cms):
    TN, FP, FN, TP = cm_vals
    plot_cm(ax, TN, FP, FN, TP, title=f"{int(r.fraction_pct)}%")
plt.show()
```



In [67]:

```
# ==================== Visualize one sample across fractions ==================
import numpy as np
import matplotlib.pyplot as plt
import torch

# --- robust checkpoint loader (same style as before) ---
def load_ckpt_any(path, device="cpu", default_in_ch=768, default_num_classes=2,
    import torch, numpy as np, torch.serialization as ts
    try:
        ck = torch.load(path, map_location=device, weights_only=True)
    except Exception:
        try:
            from numpy.core.multiarray import _reconstruct
            ts.add_safe_globals([_reconstruct, np.dtype])
            ck = torch.load(path, map_location=device, weights_only=True)
        except Exception:
            print("[warn] weights_only=False fallback (ensure you trust this fil
            ck = torch.load(path, map_location=device, weights_only=False)

    if isinstance(ck, dict) and "decoder_state" in ck:
        state = ck["decoder_state"]
        meta  = {
            "in_ch":      ck.get("in_ch", default_in_ch),
            "num_classes":ck.get("num_classes", default_num_classes),
            "img_size":   ck.get("img_size", default_img_size),
        }
    elif isinstance(ck, dict):
        state = ck
        meta  = {"in_ch": default_in_ch, "num_classes": default_num_classes, "im
    else:
        raise RuntimeError(f"Unsupported checkpoint type: {type(ck)}")
    return state, meta

@torch.no_grad()
def show_sample_across_fractions(sample_idx=25, fractions=(10,20,30,50,100), thr
    # get the underlying test dataset if only the loader exists
    try:
        ds = test_ds
    except NameError:
        ds = test_dl.dataset

    # pull one sample
    sample = ds[sample_idx]
    x0 = sample["image"].unsqueeze(0).to(device)   # [1,3,H,W]
```

```
    gt = sample["mask"].cpu().numpy().astype(np.uint8)
    rgb0 = (sample["image"].cpu().numpy().transpose(1,2,0) * std_rgb + mean_rgb)

    # gather rows for the requested fractions (keep original order)
    rows = []
    for f in fractions:
        match = df[df["fraction_pct"]==f]
        if len(match)==0:
            print(f"[skip] No checkpoint for {f}%")
            continue
        rows.append(match.iloc[0])

    # figure: one row per fraction, 3 columns (RGB, GT, Pred)
    R = len(rows)
    fig, axes = plt.subplots(R, 3, figsize=(9, 3*R))
    if R == 1: axes = np.array([axes])

    for r, row in enumerate(rows):
        # load decoder
        state, meta = load_ckpt_any(row.ckpt, device=device, default_in_ch=768,
        dec = SmallDecoder(in_ch=meta["in_ch"], num_classes=meta["num_classes"],
        dec.load_state_dict(state); dec.eval()

        # forward
        f = forward_feats(x0)                       # encoder features
        logits = dec(f)                             # [1,2,H,W]
        probs = torch.softmax(logits, dim=1)[0,1].detach().cpu().numpy()
        pred  = (probs >= thr).astype(np.uint8)

        # plot
        axes[r,0].imshow((rgb0*255).astype(np.uint8)); axes[r,0].set_title("RGB"
        axes[r,1].imshow(gt, cmap="gray", vmin=0, vmax=1); axes[r,1].set_title("
        axes[r,2].imshow(pred, cmap="gray", vmin=0, vmax=1); axes[r,2].set_title

    plt.tight_layout(); plt.show()

# ===== call it =====
show_sample_across_fractions(sample_idx=59, fractions=(10,20,30,50,100), thr=0.5
```


RGB
GT
Pred (10%)
RGB
GT
Pred (20%)
RGB
GT
Pred (30%)
RGB
GT
Pred (50%)
RGB
GT
Pred (100%)